

Final Project

Emergency Evacuation System

Mata kuliah : Struktur Data dan Pemrograman Berorientasi Objek
Kelas : A
Dosen pengampu : Hafara Firdausi, S.Kom., M.Kom
Github : https://github.com/baboey/FP_DSOOP2026/

Anggota Kelompok 6:

- 5027231007 Thio Billy Amansyah
- 5027251050 Sean Arthur Tamajaya
- 5027251096 Afriezal Suryapraba Laiasach
- 5027251116 Dian Hanna Simanjuntak

I. Deskripsi Masalah

Dalam kondisi darurat seperti kebakaran atau bencana, proses evakuasi membutuhkan sistem yang mampu menentukan jalur paling aman menuju lokasi tujuan. Setiap jalur memiliki kondisi berbeda seperti waktu tempuh, tingkat risiko, kepadatan jalur, dan tingkat asap. Oleh karena itu, dibuat Emergency Evacuation System berbasis Graph yang dapat membantu pengguna mencari rute evakuasi optimal dengan mempertimbangkan kondisi lingkungan menggunakan algoritma Dijkstra serta melakukan penelusuran area menggunakan BFS.

II. Dataset

Dataset yang digunakan pada sistem Emergency Evacuation System berupa file CSV yang berisi informasi jalur antar lokasi pada gedung. Setiap baris pada dataset merepresentasikan satu hubungan jalur antara dua lokasi yang saling terhubung. Data tersebut digunakan untuk membangun struktur graph, dimana lokasi direpresentasikan sebagai vertex dan jalur direpresentasikan sebagai edge.

Atribut	Keterangan
src	lokasi awal
dst	lokasi tujuan
time	waktu tempuh jalur
risk	tingkat risiko jalur
distance	jarak jalur
congestion	tingkat kepadatan jalur
smoke_level	tingkat asap/kondisi bahaya

III. Struktur Graph

Pada sistem Emergency Evacuation System, struktur data graph digunakan untuk merepresentasikan hubungan antar lokasi dalam gedung. Setiap lokasi ruangan direpresentasikan sebagai *vertex*, sedangkan jalur yang menghubungkan antar lokasi direpresentasikan sebagai *edge*.

Implementasi graph menggunakan *weighted undirected graph* dengan representasi *adjacency list* berbasis *linked list*. Setiap *vertex* menyimpan daftar *edge* yang terhubung ke *vertex* lain menggunakan *linked list*, sehingga hubungan antar lokasi dapat dikelola secara dinamis.

Adjacency list dipilih karena lebih efisien dibanding *adjacency matrix* pada graph yang jumlah koneksinya tidak penuh. Selain itu, penggunaan *linked list* memungkinkan penambahan dan penghapusan lokasi maupun jalur secara dinamis sesuai kondisi darurat.

IV. Struktur Tree

Struktur Tree yang kami gunakan disini adalah MinHeap, dimana memiliki fungsi untuk menyimpan ID ruangan beserta akumulasi risiko sementara dari titik awal evakuasi menuju ruangan tersebut. Untuk cara kerjanya sesuai dengan sifat dasar struktur Min-Heap, elemen dengan nilai terkecil (dalam kasus ini, ruangan dengan akumulasi risiko paling minimal) akan selalu berada di posisi paling atas (root). MinHeap ini menjadi komponen krusial untuk mempercepat performa Algoritma Dijkstra, karena ia tinggal mengambil (extract-min) node teratas dari MinHeap untuk mendapatkan ruangan dengan resiko terkecil yang belum dikunjungi.

Kenapa kok MinHeap disini bisa dikatakan sebagai Struktur data tree karena dalam aturan bentuknya sendiri Min-Heap wajib diisi secara berurutan dari atas ke bawah, dan dari kiri ke kanan. Tidak boleh ada "kursi kosong" di tingkat (level) yang atas sebelum mengisi tingkat di bawahnya. Lalu untuk aturan nilai, nilai pada sebuah Node (induk) harus selalu lebih kecil atau sama dengan nilai anak-anaknya. Sehingga nilai paling kecil di seluruh sistem akan otomatis naik dan duduk di posisi paling atas (Root).

V. Algoritma yang Digunakan

Algoritma yang kita gunakan sendiri itu ada dua yaitu Dijkstra dan BFS. Dijkstra disini digunakan untuk menemukan rute teraman dengan total resiko paling kecil saat melakukan evakuasi. Rute teraman didapatkan dengan mengevaluasi total risiko terkecil pada setiap jalur. **HeapNode digunakan untuk menyimpan ID ruangan beserta akumulasi risiko sementara** dari titik awal ke ruangan tersebut, yang nantinya akan diurutkan oleh MinHeap. Total risiko didapatkan dari penjumlahan `risk + congestion + smoke_level`.

Langkah-langkah kerja:

1. Mencari ruangan dengan resiko terkecil yang belum pernah dikunjungi.
Disini program tidak perlu mengecek ruangan satu per satu, MinHeap akan langsung menyodorkan ruangan yang paling aman di posisi paling atas.

2. Eksplorasi tetangga & relaksasi: Program akan melihat semua koridor/pintu yang menempel di ruangan tersebut, misalkan ruangan 20 terhubung ke ruangan 3 (bobot risiko koridor = 5). Program akan melakukan kalkulasi pintar Risiko baru = Risiko ruangan awal (0) + Bobot koridor (5), program akan membandingkan catatan lama dari ruangan 3 yang awalnya tidak terhitung menjadi 5 karena 5 jauh lebih kecil. Proses membandingkan dan memperbarui nilai risiko ruangan tujuan dengan nilai yang lebih kecil inilah yang dinamakan **relaksasi**.
3. Catat jejak balik: Setiap kali ada rute yang telah di relaksasi, program akan mencatat siapa "ruangan bapak/sebelumnya".
Contoh: Karena jalur 3 ditemukan lewat Ruangan 20, maka program mencatat: `parentNode[3] = 20`.
4. Semua proses di atas akan diulangi sampai ketemu ruangan safe zone yang telah kita pilih.

BFS disini digunakan untuk mencari semua jalur yang tersedia dari ruangan asal ke ruangan tujuan. Fungsinya adalah untuk memetakan kawasan tercepat berdasarkan jumlah lompatan ruangan terkecil.

Langkah-langkah kerja:

1. Antre dan Catat Titik Awal: Program memasukkan ruangan awal misal 20 ke dalam queue dan mencatat nomor 20 di dalam HashSet Visited (ruangan yang telah dikunjungi). Catatan visited ini penting agar program tidak mengunjungi ruangan yang sama dua kali.
2. Setelah ruangan tadi disimpan di visited, ruangan tersebut bakal keluar dari queue menggunakan `queue.poll()` dan akan langsung dicetak di terminal. Lalu program akan mengecek setiap edge (koridor/pintu) yang ada di ruangan 20 ([current.is](#)). Misal koridor dari ruangan 20 itu ada yang menuju ruangan 3 dan 1.
3. Lalu program akan mengecek apakah ruangan 3 dan 1 itu sudah pernah dikunjungi (visited). Karena belum program akan memasukkan ruangan 3 dan 1 ke HashSet visited dan memasukkan ruangan 3 dan 1 ke belakang Queue untuk menunggu giliran dieksplorasi pada putaran berikutnya
4. Proses diatas akan di ulang terus.

VI. Design Decision Log

Keputusan	Alternatif	Alasan	Risiko
Menggunakan Graph dengan representasi <i>adjacency list</i>	<i>Adjacency matrix</i>	<i>Adjacency list</i> lebih sesuai untuk menyimpan hubungan antar lokasi yang tidak terhubung penuh dan lebih hemat memori.	Pencarian <i>edge</i> membutuhkan traversal pada <i>linked list</i> .
Menggunakan <i>Vertex</i> dan <i>Edge</i> berbasis <i>linked</i>	Array atau struktur graph	<i>Linked list</i> memudahkan pengelolaan penambahan dan penghapusan lokasi serta	Relasi antar node lebih kompleks untuk dikelola.

<i>list</i>	bawaan Java	jalur secara dinamis.	
Menggunakan <i>weighted undirected graph</i>	Menggunakan <i>unweighted graph</i> atau <i>directed graph</i>	Jalur evakuasi memiliki nilai bobot dan dapat dilalui dua arah sehingga diperlukan graph berbobot tidak berarah.	Penyimpanan edge menjadi lebih banyak.
Menggunakan bobot gabungan <i>risk, congestion, dan smoke level</i>	Hanya menggunakan <i>risk</i>	Kondisi evakuasi lebih realistis karena mempertimbangkan beberapa faktor keamanan.	Perhitungan bobot menjadi lebih kompleks.
Menggunakan BFS untuk traversal area	DFS	BFS menggunakan <i>queue</i> sehingga dapat melakukan penelusuran lokasi secara bertahap dari titik awal.	Tidak mempertimbangkan tingkat keamanan jalur.
Menggunakan Dijkstra dengan MinHeap	BFS untuk mencari jalur	Dijkstra dapat menentukan jalur dengan total bobot keamanan terkecil, sedangkan MinHeap membantu mengambil nilai minimum secara efisien.	Implementasi algoritma lebih kompleks.
Mengimplementasikan MinHeap sendiri	PriorityQueue Java	MinHeap digunakan sebagai penerapan struktur tree dan mendukung operasi Dijkstra.	Perlu mengimplementasikan operasi <i>heapify</i> secara manual.
Menggunakan dataset CSV	Data langsung di program	CSV membuat data jalur dapat diperbarui tanpa mengubah kode program.	Kesalahan format file dapat menyebabkan gagal membaca data.
Menambahkan fitur perubahan kondisi graph (<i>deleteEdge, deleteVertex, updateEdgeWeight</i>)	Graph statis	Sistem dapat mensimulasikan kondisi darurat seperti jalur tertutup, lokasi rusak, atau perubahan kondisi jalur.	Perubahan graph dapat memengaruhi hasil rute evakuasi.

VII. Tracing Manual

Pengujian (*tracing*) manual pada algoritma pencarian jalur teraman dilakukan menggunakan algoritma Dijkstra. Parameter uji coba didefinisikan sebagai berikut:

- Titik Asal (Start Node): Ruang 0
- Titik Tujuan (End Node): Ruang 11
- Mode Perhitungan Bobot: Mode 6 (Mode Cerdas Integrasi).
- Formula Bobot Mode 6: $\text{Bobot} = \text{Risk} + \text{Congestion} + \text{Smoke Level}$.

Berdasarkan dataset yang diberikan, rute dari titik 0 menuju titik 11 memiliki penjabaran bobot jalur sebagai berikut:

- Jalur 0 → 3: $\text{Risiko}(9) + \text{Kepadatan}(3) + \text{Asap}(4) = 16$.
- Jalur 3 → 11: $\text{Risiko}(3) + \text{Kepadatan}(5) + \text{Asap}(6) = 14$.

Sebelum iterasi pencarian rute dimulai, sistem melakukan inisialisasi pada beberapa struktur data:

1. Struktur Data **totalRisiko** dan **parentNode**:
Menggunakan HashMap untuk memetakan setiap ruangan (Vertex) dengan nilai akumulasi risiko dari titik asal dan melacak ruangan sebelumnya.
2. Pemberian Nilai Awal:
Seluruh *vertex* di dalam *graph* diberikan nilai risiko tak terhingga (**Integer.MAX_VALUE**), kecuali titik asal (Ruang 0) yang diinisialisasi dengan nilai 0.
3. Antrian Prioritas:
Titik asal (0) dengan bobot risiko 0 dimasukkan ke dalam **MinHeap** untuk memulai iterasi.

Algoritma melakukan iterasi secara terus-menerus selama **MinHeap** tidak dalam kondisi kosong (**!minHeap.isEmpty()**). Proses iterasi untuk skenario ini adalah sebagai berikut:

- **Iterasi ke-1 (Ekstraksi Titik 0):**
 - Metode **extractMin()** pada **MinHeap** mengeluarkan Ruang 0 sebagai titik dengan risiko terkecil saat ini.
 - Sistem mencari referensi objek *vertex* menggunakan metode **findVertex(0)**.
 - Sistem mengevaluasi tetangga dari Ruang 0. Ditemukan Ruang 3 dengan bobot jalur sebesar 16.
 - Sistem menghitung total risiko baru: $\text{Risiko Ruang 0 (0)} + \text{Bobot Jalur (16)} = 16$.
 - Karena 16 lebih kecil dari nilai tak terhingga (batas awal), sistem memperbarui nilai **totalRisiko** Ruang 3 menjadi 16, mencatat **parentNode** Ruang 3 sebagai Ruang 0, dan memasukkan Ruang 3 ke dalam **MinHeap**.
- **Iterasi ke-2 (Ekstraksi Titik 3):**
 - Ruang 3 diekstraksi dari **MinHeap**.

- Sistem mengevaluasi tetangga dari Ruang 3. Ditemukan Ruang 11 dengan bobot jalur sebesar 14.
- Sistem menghitung total risiko baru menuju Ruang 11: Risiko Ruang 3 (16) + Bobot Jalur (14) = 30.
- Nilai **totalRisiko** untuk Ruang 11 diperbarui menjadi 30, dicatat **parentNode**-nya sebagai Ruang 3, dan dimasukkan ke dalam **MinHeap**.

Iterasi dihentikan ketika titik yang diekstraksi dari **MinHeap** sama dengan titik tujuan (Ruang 11). Setelah *break* dari *loop*, sistem memanggil metode **cetakRute** untuk membentuk hasil akhir:

1. Penyusunan Rute:
Sistem menggunakan struktur data **Stack** untuk menelusuri mundur (*backtrack*) rute dari titik tujuan menggunakan **parentNode**.
2. Eksekusi Stack:
Penelusuran berawal dari 11 → 3 → 0. Objek-objek tersebut di-*push* ke dalam Stack, dan kemudian di-*pop* untuk menghasilkan cetakan dengan urutan maju: 0 → 3 → 11.

VIII. Screenshot Hasil Program

Menu program:

```
=====
1. Tampilkan Struktur Peta Gedung (Graph)
2. Cari Rute Evakuasi Teraman (Dijkstra - Mode Cerdas)
3. Simulasi Jalur Blokir / Runtuh (Hapus Edge)
4. Telusuri Seluruh Area (BFS Traversal)
5. Hapus Ruang (Delete Vertex)
6. Ubah Bobot Koridor (Update Edge)
7. Cari Ruang dengan Prefix
8. Keluar
Pilih menu (1-8):
```

Opsi 1 menampilkan Struktur Peta Gedung:

```

--- PETA STRUKTUR GEDUNG (ADJACENCY LIST) ---
[24]
24 - 17 (21)
-----
[21]
21 - 12 (6)
-----
[19]
19 - 8 (17)
-----
[18]
18 - 6 (24)
-----
[22]
22 - 20 (17)
22 - 17 (19)
-----
[15]
15 - 5 (17)
15 - 13 (19)
15 - 12 (8)
-----
[12]

```

Opsi 2 Cari Rute Evakuasi Teraman:

```

Pilih menu (1-8): 2

Masukkan lokasi posisi Anda saat ini (Angka): 20
Masukkan nomor titik aman / Safe Zone (Angka): 6

Menghitung rute evakuasi memproses Min-Heap...
Rute Evakuasi Teraman (Dijkstra): 20 -> 22 -> 17 -> 6
Total Bobot Risiko Keamanan: 43

```

Opsi 3 Simulasi jika ada jalur yang terblokir:

```

--- SIMULASI KEADAAN DARURAT (JALUR TERTUTUP) ---
Masukkan titik asal jalan yang runtuh: 22
Masukkan titik tujuan jalan yang runtuh: 6
JALUR ANTARA 22 DAN 6 TELAH DITUTUP!

```

Opsi 4 Telusuri seluruh area menggunakan BFS:

```

Pilih menu (1-8): 4

Masukkan titik awal penelusuran BFS (Angka): 20

Hasil Penelusuran BFS:
20 22 11 2 17 13 9 24 8 6 15 10 19 18 5 12 7 1 23 21 14 16 0 3 4

```

Opsi 5 Hapus Ruangan jika Ruangan hancur total:

```

Pilih menu (1-8): 5

--- HAPUS RUANGAN TOTAL ---
Masukkan nomor ruangan yang hancur/runtuh total: 20
Ruangan 20 dan seluruh akses jalurnya telah dihapus dari peta!

```

Opsi 6:

```

--- UBAH BOBOT KORIDOR ---
Masukkan nomor ruangan asal: 19
Masukkan nomor ruangan tujuan: 9
Masukkan nilai bobot risiko baru (Angka): 12
Bobot jalur antara 19 dan 9 berhasil diubah menjadi: 12

```

Before:

```

[19]
19 - 9 (9)
19 - 13 (24)
19 - 0 (15)
-----

```

After:

```

[19]
19 - 9 (12)
19 - 13 (24)
19 - 0 (15)
-----

```

Opsi 7 Mencari ruangan berdasarkan Prefix:

```

--- CARI RUANGAN BERDASARKAN PREFIX ---
Masukkan angka prefix/awalan lantai yang dicari: 1
Hasil pencarian ruangan dengan awalan '1':
- Ruangan [ID: 19]
- Ruangan [ID: 18]
- Ruangan [ID: 15]
- Ruangan [ID: 12]
- Ruangan [ID: 13]
- Ruangan [ID: 11]
- Ruangan [ID: 10]
- Ruangan [ID: 17]
- Ruangan [ID: 16]
- Ruangan [ID: 1]
- Ruangan [ID: 14]

```

IX. Analisis Kompleksitas

Analisis Kompleksitas Waktu (Time Complexity)

Kompleksitas waktu mengukur seberapa lama algoritma berjalan berdasarkan ukuran data input:

1. Inisialisasi Nilai Risiko:

Sistem melakukan perulangan *for* untuk mengisi setiap *vertex* pada struktur Graph ke dalam **HashMap totalRisiko**. Iterasi ini dilakukan sebanyak jumlah ruangan, sehingga membutuhkan kompleksitas waktu **$O(V)$** .

2. Operasi Pencarian Vertex (*Lookup*):

Metode pencarian jalur melakukan panggilan **graph.findVertex(u)** di dalam setiap iterasi *while*. Berdasarkan arsitektur sistem, struktur Graf menggunakan implementasi *Linked List* manual melalui atribut **ls** dan **next**. Pencarian *vertex* dalam *linked list* ini membutuhkan kompleksitas waktu seburuk-buruknya **$O(V)$** .

3. Operasi Pembaruan Rute (Min-Heap):

Struktur data **MinHeap** menggunakan perulangan rekursif melalui fungsi **heapifyUp** dan **heapifyDown** saat proses penyisipan (**insert**) dan ekstraksi (**extractMin**). Ini memberikan kompleksitas waktu logaritmik **$O(\log V)$** untuk setiap operasi. Pembaruan jalur atau tetangga terjadi secara maksimal sebanyak **E** kali di sepanjang jalannya algoritma.

Kesimpulan Kompleksitas Waktu:

Secara akumulatif, algoritma ini membutuhkan pemrosesan sebanyak **$O(V)$** untuk inisialisasi, dan secara asimptotik melakukan pencarian **$O(V)$** dan operasi logaritmik **$O(\log V)$** sebanyak **E** kali iterasi. Oleh karena itu, kompleksitas waktu *worst-case* yang dihasilkan adalah $O(V + E \times V + E \log V)$, yang secara asimptotik disederhanakan menjadi $O(E \times V)$ akibat sifat pencarian linear pada struktur dasar graf.

Analisis Kompleksitas Ruang (*Space Complexity*)

Kompleksitas ruang mengukur alokasi memori (*RAM*) yang dikonsumsi oleh algoritma selama proses eksekusi berlangsung:

1. Alokasi Variabel *State* Rute:

Penggunaan dua instansi **HashMap**, yaitu **totalRisiko** dan **parentNode**, dirancang untuk menyimpan data masing-masing sebanyak jumlah *vertex* di dalam peta. Hal ini memakan alokasi memori berorde $O(V)$.

2. Alokasi *Min-Heap* Antrean:

Implementasi **MinHeap** tidak menerapkan fitur *decrease-key* secara *in-place*, sehingga setiap ada penemuan rute baru yang lebih cepat menuju titik yang sama, sistem akan menambah objek **HeapNode** baru ke dalam **ArrayList**. Pada kasus terburuk (*dense graph*), **MinHeap** bisa menyimpan elemen sebanyak jumlah koneksi yang ada, yang membutuhkan ruang memori sebesar $O(E)$.

3. Alokasi Stack Rekonstruksi:

Variabel lokal **Stack path** memuat rentetan rute yang harus dilewati. Jika rute membentuk sebuah rantai utuh melewati semua titik, ia membutuhkan memori maksimum $O(V)$.

Kesimpulan Kompleksitas Ruang:

Penjumlahan atas seluruh ruang memori di atas menghasilkan nilai maksimum pada memori utama yang dibutuhkan, yakni **$O(V + E)$** .

Kesimpulan Analisis

Penggunaan struktur *Adjacency List* menggunakan *custom Linked List* memudahkan dalam manipulasi dinamis, namun menyebabkan terjadinya pencarian *Vertex* yang linear ($O(V)$) dalam proses utamanya. Walaupun implementasi *MinHeap* telah bekerja secara optimal pada orde $O(\log V)$, kompleksitas waktu akhir sistem terpengaruh oleh pencarian objek tersebut dan menjadi $O(E \times V)$. Di sisi lain, algoritma dieksekusi secara efisien secara memori, hanya memakan ruang $O(V + E)$ berkat penggunaan *HashMap* dan *ArrayList* yang adaptif pada *MinHeap*.

X. What-If Analysis

What-if 1	Bagaimana jika tingkat asap pada suatu jalur meningkat?
Nilai <code>smoke_level</code> akan meningkatkan bobot jalur sehingga algoritma Dijkstra dapat menghindari jalur tersebut dan memilih jalur lain dengan kondisi lebih aman.	
What-if 2	Bagaimana jika sebuah ruangan mengalami kerusakan total?
Sistem dapat menggunakan fitur <code>deleteVertex</code> untuk menghapus lokasi beserta seluruh koneksi jalurnya dari graph. Setelah itu pencarian rute dilakukan kembali untuk mendapatkan alternatif.	
What-if 3	Bagaimana jika kondisi jalur berubah selama proses evakuasi?
Sistem dapat memperbarui bobot edge menggunakan fitur <code>updateEdgeWeight</code> sehingga hasil jalur evakuasi dapat menyesuaikan kondisi terbaru.	

XI. Kesimpulan

Emergency Evacuation System berhasil mengimplementasikan struktur data Graph menggunakan *adjacency list* berbasis *linked list* untuk merepresentasikan lokasi dan jalur evakuasi. Setiap jalur memiliki bobot berdasarkan kondisi lingkungan seperti risiko, kepadatan, dan tingkat asap. Algoritma BFS digunakan untuk melakukan traversal area, sedangkan algoritma Dijkstra dengan bantuan *MinHeap* digunakan untuk mencari jalur evakuasi dengan tingkat keamanan terbaik. Selain itu, sistem dilengkapi fitur simulasi perubahan kondisi seperti penutupan jalur, penghapusan lokasi, perubahan bobot jalur, dan pencarian lokasi untuk meningkatkan fleksibilitas sistem dalam menghadapi kondisi darurat.